Documentation for the incompressible Navier-Stokes Matlab codes

William D. Henshaw[a,1], Gabriel Weredyk[a]

[a] *Department of Mathematical Sciences, Rensselaer Polytechnic Institute, Troy, NY 12180, USA*

Abstract

Here is documentation and notes for the incompressible Navier-Stokes Matlab codes.

Contents

1 The MATLAB code **2**
1.1 Description of Files . . . 2
1.2 Description of input parameters to ins.m . . . 2
1.2.1 Solution Description Parameters . . . 2
1.2.2 Physical Parameters . . . 2
1.2.3 Time & Timestepping Parameters . . . 2
1.2.4 Grid & Geometry Parameters . . . 2
1.2.5 Grid Motion Parameters . . . 2
1.2.6 Artificial Terms Parameters . . . 2
1.2.7 Optimization & Computation Parameters . . . 2
1.2.8 Printing Parameters . . . 2
1.2.9 Plotting Parameters . . . 2

2 Introduction, governing equations, boundary conditions **2**
2.1 Boundary conditions . . . 3

3 Numerical approach **5**
3.1 Discrete approximations in space . . . 5
3.2 Time-stepping algorithms . . . 5
3.3 Free surface evolution . . . 5

4 Numerical Results **6**
4.1 Manufactured solutions . . . 6
4.2 Poiseuille Flow . . . 8
4.3 Travelling Wave of a Linearized Surface . . . 11
4.4 Taylor Green vortex . . . 12
4.5 Shear flow in a periodic channel . . . 13

5 Viscous gravity-capillary travelling wave **14**
5.1 Problem statement . . . 14
5.2 Hyperbolic basis shifted to the bottom wall . . . 14
5.3 Dispersion relation . . . 15
5.4 Solving for ω and the coefficients . . . 15
5.5 Comparison with the thesis constants . . . 16
5.6 Matlab implementation . . . 16

Email addresses: henshw@rpi.edu (William D. Henshaw), weredg@rpi.edu (Gabriel Weredyk)

1. The MATLAB code

1.1. Description of Files

ins.m	Main routine .
runChecks.m	Run regression tests.
runGridConvergence.m	Run a grid convergence test.
advanceAdams.m	Adams time-stepping.
advanceIM.m	IMEX time-stepping.
advancePC.m	Predictor-Corrector time-stepping.
applyBoundaryConditions.m	apply (explicit) boundary conditions
pressureEquation.m	form pressure equation and solve
evalMap.m	eval the grid mapping and derivatives
formImplicitTimeSteppingMatrix.m	form systems for implicit time-stepping of u or v
formImplicitTimeSteppingMatrixCombined.m	form systems for implicit time-stepping for (u, v)
getGrid.m	get the grid for moving grid problems
getTimeStep.m	compute the time-step
solveImplicitTimeStep.m	solve the implicit time-step equations of u or v
solveImplicitTimeStepCombined.m	solve the implicit time-step equations for (u, v)
getInitialConditions.m	assign initial conditions
getUt.m	evaluate momentum equations
getTimeStep.m	determine the time-step Δt
plotSolution.m	plot solution contours and streamlines
plotStreamLines.m	stream line plotter

Table 1: Selected ins Matlab files.

1.2. Description of input parameters to ins.m

The main routine of the code in the repository is ran through `ins.m`, and parameters are specified on the command line

1.2.1. Solution Description Parameters
1.2.2. Physical Parameters
1.2.3. Time & Timestepping Parameters
1.2.4. Grid & Geometry Parameters
1.2.5. Grid Motion Parameters
1.2.6. Artificial Terms Parameters
1.2.7. Optimization & Computation Parameters
1.2.8. Printing Parameters
1.2.9. Plotting Parameters

2. Introduction, governing equations, boundary conditions

The `ins.m` Matlab code solves the incompressible Navier-Stokes equations in velocity-pressure form,

$$\partial_t \mathbf{u} + (\mathbf{u}\cdot\nabla)\mathbf{u} + \frac{1}{\rho}\nabla p = \nu\Delta\mathbf{u} + \mathbf{g} + \mathbf{f}(\mathbf{x},t), \quad \mathbf{x}\in\Omega,\ t>0, \tag{1a}$$

$$\frac{1}{\rho}\Delta p = -\nabla\mathbf{u} : \nabla\mathbf{u} + \nabla\cdot\mathbf{f}, \qquad \mathbf{x}\in\Omega,\ t>0, \tag{1b}$$

$$\mathbf{u}(\mathbf{x},0) = \mathbf{u}_0(\mathbf{x}) \qquad \mathbf{x}\in\Omega, \tag{1c}$$

$$\nabla\cdot\mathbf{u}(\mathbf{x},0) = 0 \qquad \mathbf{x}\in\partial\Omega, \tag{1d}$$

where $\nu = \mu/\rho$ is the kinematic viscosity coefficient, $\mathbf{g}$ is the gravity vector and $\mathbf{f}(\mathbf{x},t)$ is a body forcing. These equations are augmented with appropriate boundary conditions as discussed below.

For problems with surface tension, the traction free boundary condition for the pressure is

$$-p = \mathbf{n}^T\boldsymbol{\tau}\mathbf{n} = \gamma\kappa \tag{2}$$

where $\tau_{ij} = \mu(\partial_{x_j} u_i + \partial_{x_i} u_j)$, γ is the coefficient of surface tension and κ is the curvature. Non-dimensional numbers associated with the surface tension are

$$\text{We} = \frac{\rho v^2 L}{\gamma}, \qquad \text{Weber number: ratio of inertial to surface tension forces,} \tag{3a}$$

$$\text{Ca} = \frac{\mu v}{\gamma} \qquad \text{Capillary number: ratio of viscous to surface tension forces,} \tag{3b}$$

$$\text{Bo} = \frac{\Delta\rho\, g L^2}{\gamma} \qquad \text{Bond number: ratio of buoyancy to surface tension forces,} \tag{3c}$$

$$\text{Oh} = \frac{\mu}{\sqrt{\rho\gamma L}} \qquad \text{Ohnesorge number: relates viscous to inertial and surface tension forces,} \tag{3d}$$

where v is the velocity scale, L the length scale, and where $\Delta\rho$ is the density difference between the two fluids (or ρ for a free surface).

2.1. Boundary conditions

Let $\mathbf{n}(\mathbf{x})$ denote the outward unit normal vector to a point $\mathbf{x}$ on a boundary. Let $\mathbf{t}$ denote the corresponding unit tangent vector. The stress tensor is $\boldsymbol{\sigma} = -p\mathbf{I} + \boldsymbol{\tau}$ with $\boldsymbol{\tau} = \mu(\nabla\mathbf{u} + (\nabla\mathbf{u})^T)$. They have components

$$\sigma_{ij} = -p\delta_{ij} + \mu(\partial_{x_j} u_i + \partial_{x_i} u_j), \tag{4a}$$

$$\tau_{ij} = \mu(\partial_{x_j} u_i + \partial_{x_i} u_j). \tag{4b}$$

In two dimensions,

$$\boldsymbol{\sigma} = \begin{bmatrix} -p + 2\mu u_x & \mu(u_y + v_x) \\ \mu(u_y + v_x) & -p + 2\mu v_y \end{bmatrix} \tag{5}$$

Here are the physical boundary conditions. Numerically, additional compatibility conditions will be used.

No-slip wall or inflow.

$$\mathbf{u}(\mathbf{x}, t) = \mathbf{g}(\mathbf{x}, t), \quad \mathbf{x} \in \partial\Omega,\ t > 0, \tag{6a}$$

$$\nabla \cdot \mathbf{u}(\mathbf{x}, t) = 0, \quad \mathbf{x} \in \partial\Omega,\ t > 0. \tag{6b}$$

Slip wall.

$$\mathbf{n}^T \mathbf{u}(\mathbf{x}, t) = 0 \quad \mathbf{x} \in \partial\Omega,\ t > 0, \tag{7a}$$

$$\mathbf{t}^T \boldsymbol{\sigma}\mathbf{n} = 0, \quad \mathbf{x} \in \partial\Omega,\ t > 0, \tag{7b}$$

$$\nabla \cdot \mathbf{u}(\mathbf{x}, t) = 0 \quad \mathbf{x} \in \partial\Omega,\ t > 0. \tag{7c}$$

With explicit time-stepping, the slipwall condition (7a) can be imposed by first updating the values on the boundary using the interior PDE and then using (7a) to set the normal component to zero,

$$\mathbf{u} \leftarrow \mathbf{u} - (\mathbf{n}^T \mathbf{u})\mathbf{n}. \tag{8}$$

With implicit time-stepping, the slipwall condition (7a) can be combined with the interior PDE to give the two equations (thought of as determining the boundary points)

$$\mathbf{n}\mathbf{n}^T \mathbf{u}(\mathbf{x}, t) + (I - \mathbf{n}\mathbf{n}^T)(\text{PDE}) = 0. \tag{9}$$

The slipWall conditions (7b) and (7b) can be written as (see comments below for traction BC)

$$(\nabla \cdot \mathbf{u}(\mathbf{x}, t))\mathbf{n} + (I - \mathbf{n}\mathbf{n}^T)\boldsymbol{\sigma}\mathbf{n} = 0. \tag{10}$$

which re thought of as determining the ghost points.

Outflow.

$$a_0 p(\mathbf{x}, t) + a_1 \partial_n p(\mathbf{x}, t) = g(\mathbf{x}, t), \quad \mathbf{x} \in \partial\Omega,\ t > 0, \tag{11a}$$

$$\partial_n^2(\mathbf{t}\cdot\mathbf{u}) \qquad \mathbf{x}\in\partial\Omega,\ t>0, \tag{11b}$$
$$\nabla\cdot\mathbf{u}(\mathbf{x},t)=0 \qquad \mathbf{x}\in\partial\Omega,\ t>0. \tag{11c}$$

Pressure inflow.

$$p(\mathbf{x},t)=g_1(\mathbf{x},t), \quad \mathbf{x}\in\partial\Omega,\ t>0, \tag{12a}$$
$$\mathbf{t}\cdot\mathbf{u}=g_2 \qquad \mathbf{x}\in\partial\Omega,\ t>0, \tag{12b}$$
$$\nabla\cdot\mathbf{u}(\mathbf{x},t)=0 \qquad \mathbf{x}\in\partial\Omega,\ t>0. \tag{12c}$$

Traction (or free surface). The traction (or free surface) boundary condition is

$$\boldsymbol{\sigma}\mathbf{n}=\gamma\kappa(\mathbf{x})\mathbf{n}, \quad \mathbf{x}\in\partial\Omega,\ t>0, \tag{13a}$$
$$\nabla\cdot\mathbf{u}(\mathbf{x},t)=0 \quad \mathbf{x}\in\partial\Omega,\ t>0. \tag{13b}$$

where γ is the coefficient of surface tension and κ is the curvature. For a surface in 3D parameterized as $\mathbf{y}(r,s)$, the curvature is

$$\kappa=\frac{\Big(\|\mathbf{y}_r\|^2\mathbf{y}_{ss}-2(\mathbf{y}_r\cdot\mathbf{y}_s)\mathbf{y}_{rs}+\|\mathbf{y}_s\|^2\mathbf{y}_{rr}\Big)\cdot\mathbf{n}}{\|\mathbf{y}_r\|^2\|\mathbf{y}_s\|^2-(\mathbf{y}_r\cdot\mathbf{y}_s)^2}. \tag{13c}$$

For a curve $\mathbf{y}(r)$ in 2D this reduces to

$$\kappa=\frac{\mathbf{y}_{rr}\cdot\mathbf{n}}{\|\mathbf{y}_r\|^2}. \tag{13d}$$

The boundary condition $\boldsymbol{\sigma}\mathbf{n}=\gamma\kappa(\mathbf{x})\mathbf{n}$ consist of the scalar equations $\mathbf{n}^T\boldsymbol{\sigma}\mathbf{n}=\gamma\kappa(\mathbf{x})$ and $\mathbf{t}^T\boldsymbol{\sigma}\mathbf{n}=0$ which are (with summation convection)

$$-p+n_i\mu(\partial_{x_j}u_i+\partial_{x_i}u_j)n_j=\gamma\kappa(\mathbf{x}), \quad \mathbf{x}\in\partial\Omega,\ t>0, \tag{13e}$$
$$\mu\, t_i(\partial_{x_j}u_i+\partial_{x_i}u_j)n_j=0, \qquad \mathbf{x}\in\partial\Omega,\ t>0. \tag{13f}$$

The boundary condition for the pressure on a traction boundary comes from the normal component of (13a),

$$\mathbf{n}^T\boldsymbol{\sigma}\mathbf{n}=\gamma\kappa(\mathbf{x}). \tag{14}$$

In two dimensions

$$\mathbf{n}^T\boldsymbol{\sigma}\mathbf{n}=-p+2\mu\Big(u_xn_1^2+v_yn_2^2+(u_y+v_x)n_1n_2\Big). \tag{15}$$

The boundary conditions used for the velocity on a traction boundary are $\nabla\cdot\mathbf{u}=0$ and $\mathbf{t}^T\boldsymbol{\tau}\mathbf{n}=0$. These can be combined as

$$(\nabla\cdot\mathbf{u})\mathbf{n}+\mathbf{t}(\mathbf{t}^T\boldsymbol{\tau}\mathbf{n})=0. \tag{16}$$

From $I=\mathbf{n}\mathbf{n}^T+\mathbf{t}\mathbf{t}^T$, it follows that $\mathbf{t}\mathbf{t}^T=I-\mathbf{n}\mathbf{n}^T$ and thus (16) can be written as

$$(\nabla\cdot\mathbf{u})\mathbf{n}+(I-\mathbf{n}\mathbf{n}^T)\boldsymbol{\tau}\mathbf{n}=0. \tag{17}$$

The form in (17) is convenient since it does not require forming tangent vectors. In addition, it is a good form to use when constucting the implicit time-stepping matrix to ensure that the matrix has non-zero diagonal entries.

3. Numerical approach

3.1. Discrete approximations in space

The discrete approach is based on the methods described in [1–3].

The method-of-lines approximation keeps time continuous and discretizes in space. Let $\mathbf{U}_\mathbf{j}(t) \approx \mathbf{u}(\mathbf{x}_\mathbf{j}, t)$, and $P_\mathbf{j}(t) = p(\mathbf{x}_\mathbf{j}, t)$. The discretization of the interior equations takes the form

$$\frac{d}{dt}\mathbf{U}_\mathbf{j} + (\mathbf{U}_\mathbf{j} \cdot \nabla_h)\mathbf{U}_\mathbf{j} + \frac{1}{\rho}\nabla_h P_\mathbf{j} = \nu\Delta_h \mathbf{U}_\mathbf{j} - \mathcal{D}_h^{(2)}\mathbf{U}_\mathbf{j}, \qquad \mathbf{j} \in \Omega_h, \tag{18a}$$

$$\Delta_h P_\mathbf{j} = \nabla_h \mathbf{U}_\mathbf{j} : \nabla_h \mathbf{U}_\mathbf{j} + \frac{c_d}{\Delta t}\nabla_h \cdot \mathbf{U}_\mathbf{j}, \qquad \mathbf{j} \in \Omega_h, \tag{18b}$$

where ∇_h and Δ_h denote discrete approximations to ∇ and Δ, respectively. The second-order artificial dissipation operator $\mathcal{D}_h^{(2)}$ is

$$\mathcal{D}_h^{(2)}\mathbf{U}_\mathbf{j} \stackrel{\text{def}}{=} (a_1 + a_2\|\nabla_h \mathbf{U}_\mathbf{j}\|_1)\Big((-\Delta x^2 D_{+x}D_{-x}) + (-\Delta y^2 D_{+y}D_{-y})\Big), \tag{18c}$$

$$\|\nabla_h \mathbf{U}_\mathbf{j}\|_1 = |D_{0x}V_{1,\mathbf{j}}| + |D_{0x}V_{2,\mathbf{j}}| + |D_{0y}V_{1,\mathbf{j}}| + |D_{0y}V_{2,\mathbf{j}}|, \tag{18d}$$

while c_d is the coefficient of the divergence damping.

3.2. Time-stepping algorithms

Time-stepping algorithms

AB2 A second-order accurate Adams-Bashforth scheme

PC2 A second-order accurate predictor-corrector scheme

IM2 A second-order accurate Implicit-explicit (IMEX) scheme.

3.3. Free surface evolution

Let $\boldsymbol{\eta}(s,t)$ denote the x-y coordinates of the free surface as a function of s and t. Here s is some parameterization of the surface. The free surface satisfies

$$\partial_t \boldsymbol{\eta} = \mathbf{u}(\boldsymbol{\eta}(s,t),t). \tag{19}$$

Here is a predictor corrector time-stepping algorithm to update the free surface from time t^n to t^{n+1}. The predictor step is a second-order accurate Adams-Bashforth scheme to compute $\eta_i^p \approx \eta^{n+1}$,

$$\boldsymbol{\eta}_i^p = \boldsymbol{\eta}_i^n + \Delta t\Big(\frac{3}{2}\mathbf{U}_\mathbf{i}^n - \frac{1}{2}\mathbf{U}_\mathbf{i}^{n-1}\Big). \tag{20}$$

After the fluid variables have been updated to t^{n+1}, the free-surface positions are corrected with a second-order accurate Adams-Moulton scheme,

$$\boldsymbol{\eta}_i^{n+1} = \boldsymbol{\eta}_i^n + \Delta t\Big(\frac{1}{2}\mathbf{U}_\mathbf{i}^{n+1} + \frac{1}{2}\mathbf{U}_\mathbf{i}^n\Big). \tag{21}$$

4. Numerical Results

4.1. Manufactured solutions

The Matlab code `runGridConvergence.m` can be used to run grid convergence tests and create a LaTeX table. Before considering exact solutions of Incompressible Navier Stokes flow, we can consider manufacturing solutions of the form

$$u = k_y \alpha \cos(k_x x)\cos(k_y y)\cos(k_t t) \tag{22a}$$
$$v = k_x \alpha \sin(k_x x)\sin(k_y y)\cos(k_t t) \tag{22b}$$
$$p = \cos(k_x x)\sin(k_y y)\cos(k_t t) \tag{22c}$$

Where α is some initial velocity amplitude. Note that despite requiring nuanced forcing functions to have these equations satisfy INS, solutions of this form are divergence free and pressure is harmonic.

Here is an example of the convergence rates for our three different time-stepping schemes , (for the implicit method we specify the Δt to use on the coarsest grid, and this is halved each resolution)

```
runGridConvergence -ts=ab2 -tf=.25 -ms=trig -numResolutions=4 -bcs=pppp
```

INS: ts=ab2, BC=pppp, Map=Cartesian, Motion=none, MS=trig									
N	Δt	E_p	r	E_u	r	E_v	r	$\lvert\nabla \cdot \mathbf{u}\rvert$	r
10	1.09e-02	5.90e-02		2.64e-03		2.71e-03		3.04e-02	
20	2.81e-03	1.58e-02	3.7	4.82e-04	5.5	4.82e-04	5.6	2.19e-03	13.9
40	7.02e-04	1.89e-03	8.3	1.20e-04	4.0	1.20e-04	4.0	2.49e-05	88.0
80	1.76e-04	9.83e-04	1.9	3.00e-05	4.0	3.00e-05	4.0	8.63e-06	2.9

```
runGridConvergence -ts=pc2 -tf=.25 -ms=trig -numResolutions=4 -bcs=pppp
```

INS: ts=pc2, BC=pppp, Map=Cartesian, Motion=none, MS=trig									
N	Δt	E_p	r	E_u	r	E_v	r	$\lvert\nabla \cdot \mathbf{u}\rvert$	r
10	1.92e-02	4.42e-02		2.76e-03		2.84e-03		3.19e-02	
20	4.90e-03	1.16e-02	3.8	4.82e-04	5.7	4.82e-04	5.9	2.24e-03	14.3
40	1.23e-03	2.91e-03	4.0	1.20e-04	4.0	1.20e-04	4.0	1.41e-04	15.8
80	3.08e-04	7.27e-04	4.0	3.00e-05	4.0	3.00e-05	4.0	8.90e-06	15.9

```
runGridConvergence -ts=im2 -tf=.25 -ms=trig -numResolutions=3 -bcs=pppp -dtMax=.01
```

INS: ts=im2, BC=pppp, Map=Cartesian, Motion=none, MS=trig									
N	Δt	E_p	r	E_u	r	E_v	r	$\lvert\nabla \cdot \mathbf{u}\rvert$	r
10	1.00e-02	4.52e-02		1.94e-03		1.92e-03		1.74e-02	
20	5.00e-03	1.16e-02	3.9	4.82e-04	4.0	4.82e-04	4.0	2.28e-03	7.6
40	2.50e-03	2.90e-03	4.0	1.20e-04	4.0	1.20e-04	4.0	2.87e-04	8.0

We can also test the manufactured solutions against different configurations of boundary conditions to see that they are in working order as well:

```
runGridConvergence -ts=pc2 -tf=.25 -ms=trig -numResolutions=4 -bcs=nnnn
```

INS: ts=pc2, BC=nnnn, Map=Cartesian, Motion=none, MS=trig									
N	Δt	E_p	r	E_u	r	E_v	r	$\lvert\nabla \cdot \mathbf{u}\rvert$	r
10	1.92e-02	4.46e-02		6.31e-03		4.17e-03		4.00e-02	
20	4.90e-03	1.16e-02	3.8	4.71e-04	13.4	3.54e-04	11.8	3.28e-03	12.2
40	1.23e-03	2.92e-03	4.0	1.17e-04	4.0	6.59e-05	5.4	2.31e-04	14.2
80	3.08e-04	7.41e-04	3.9	2.92e-05	4.0	1.57e-05	4.2	1.79e-05	12.9

```
runGridConvergence -ts=pc2 -tf=.25 -ms=trig -numResolutions=4 -bcs=ssss
```

INS: ts=pc2, BC=ssss, Map=Cartesian, Motion=none, MS=trig									
N	Δt	E_p	r	E_u	r	E_v	r	$\lvert\nabla \cdot \mathbf{u}\rvert$	r
10	1.92e-02	5.10e-02		2.25e-02		2.06e-02		3.22e-02	
20	4.90e-03	1.25e-02	4.1	3.99e-03	5.6	2.81e-03	7.3	2.30e-03	14.0
40	1.23e-03	3.01e-03	4.1	6.31e-04	6.3	4.30e-04	6.5	1.54e-04	14.9
80	3.08e-04	7.41e-04	4.1	1.04e-04	6.0	8.07e-05	5.3	1.01e-05	15.3

```
runGridConvergence -ts=pc2 -tf=.25 -ms=trig -numResolutions=4 -bcs=IIII
```

INS: ts=pc2, BC=IIII, Map=Cartesian, Motion=none, MS=trig									
N	Δt	E_p	r	E_u	r	E_v	r	$\lvert\nabla \cdot \mathbf{u}\rvert$	r
10	1.92e-02	4.46e-02		3.32e-02		4.13e-02		6.03e-02	
20	4.90e-03	1.18e-02	3.8	1.01e-02	3.3	7.27e-03	5.7	8.80e-03	6.9
40	1.23e-03	3.05e-03	3.9	2.59e-03	3.9	1.73e-03	4.2	1.16e-03	7.6
80	3.08e-04	7.80e-04	3.9	6.46e-04	4.0	4.42e-04	3.9	1.47e-04	7.9

4.2. Poiseuille Flow

Consider a fluid in Plane Poiseuille Flow over $[0,1]^2$ between two no-slip walls at $y = 0$ and $y = 1$ going from pressure p_{in} to p_{out} with $p_{\text{in}} > p_{\text{out}}$. We would define this flow by the following boundary conditions:

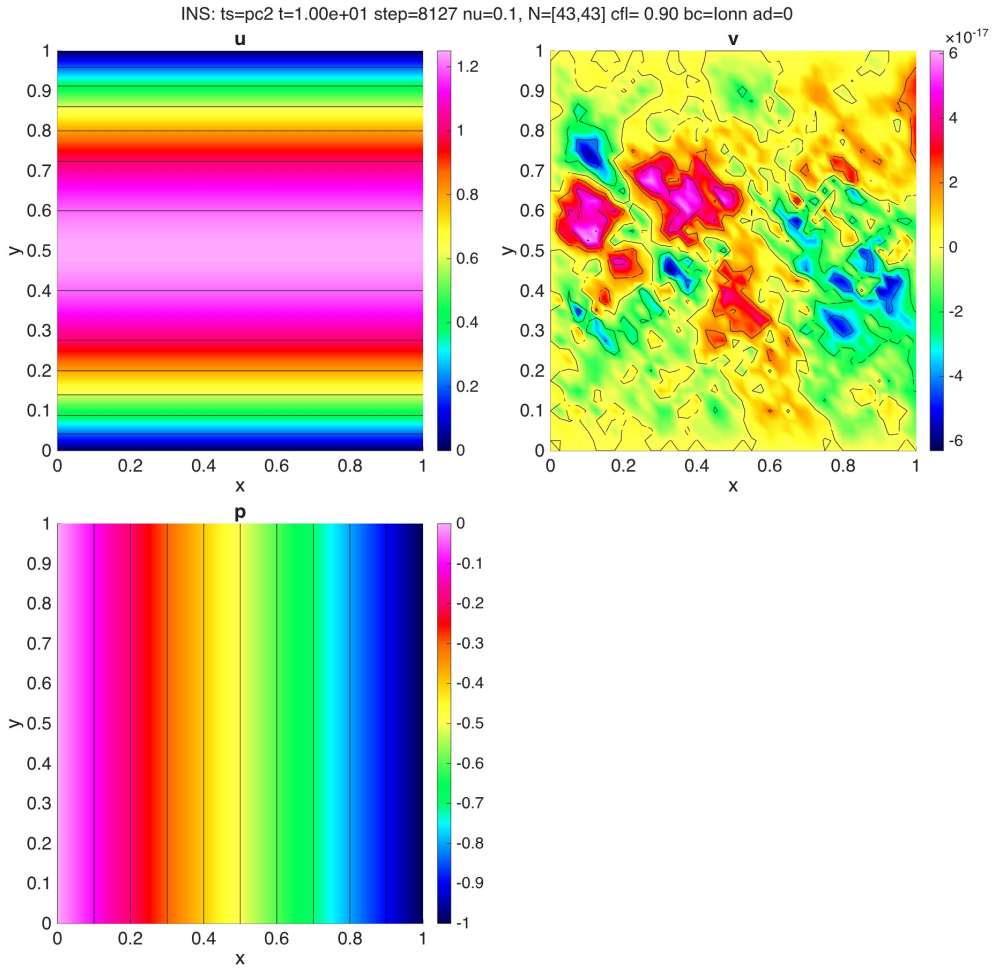
$$\mathbf{u}(x,0,t) = \mathbf{u}(x,1,t) = 0 \tag{23a}$$
$$p(0,y,t) = p_{\text{in}}, \qquad v(0,y,t) \equiv 0 \tag{23b}$$
$$p(1,y,t) = p_{\text{out}}, \qquad v(0,y,t) \equiv 0 \tag{23c}$$

The fluid then obeys a steady parabolic profile along the flow direction given by

$$u = U(y) = \frac{p_{\text{in}} - p_{\text{out}}}{2\mu} y(1-y) \tag{24a}$$
$$v \equiv 0 \tag{24b}$$
$$p = (p_{\text{out}} - p_{\text{in}})x + p_{\text{in}} \tag{24c}$$

While we can define the Poiseuille flow as an exact solution for the code to follow, we can also see that defining the initial conditions and appropriate boundary conditions, we are able to get the desired solution to. We can do so in the code by specifying

```
ins -ts=pc2 -tf=10 -N0=40 -nu=0.1
  -ic=zero -bcs=lonn
  -pressureInflowValue=0 -uInflow=0 -outflowPressureCoeffp=1 -outflowPressureCoeffpn=0 -pOutflow=-1;
```



Moreover, given the Reynolds number of the flow is not large, the plane Poiseuille flow is a stable steady-state of the INS equations, meaning if we perturb the Poiseuille flow with some other velocity profile $\mathbf{u}_p(x, y)$ that also satisfies the boundary conditions (23) at $t = 0$:

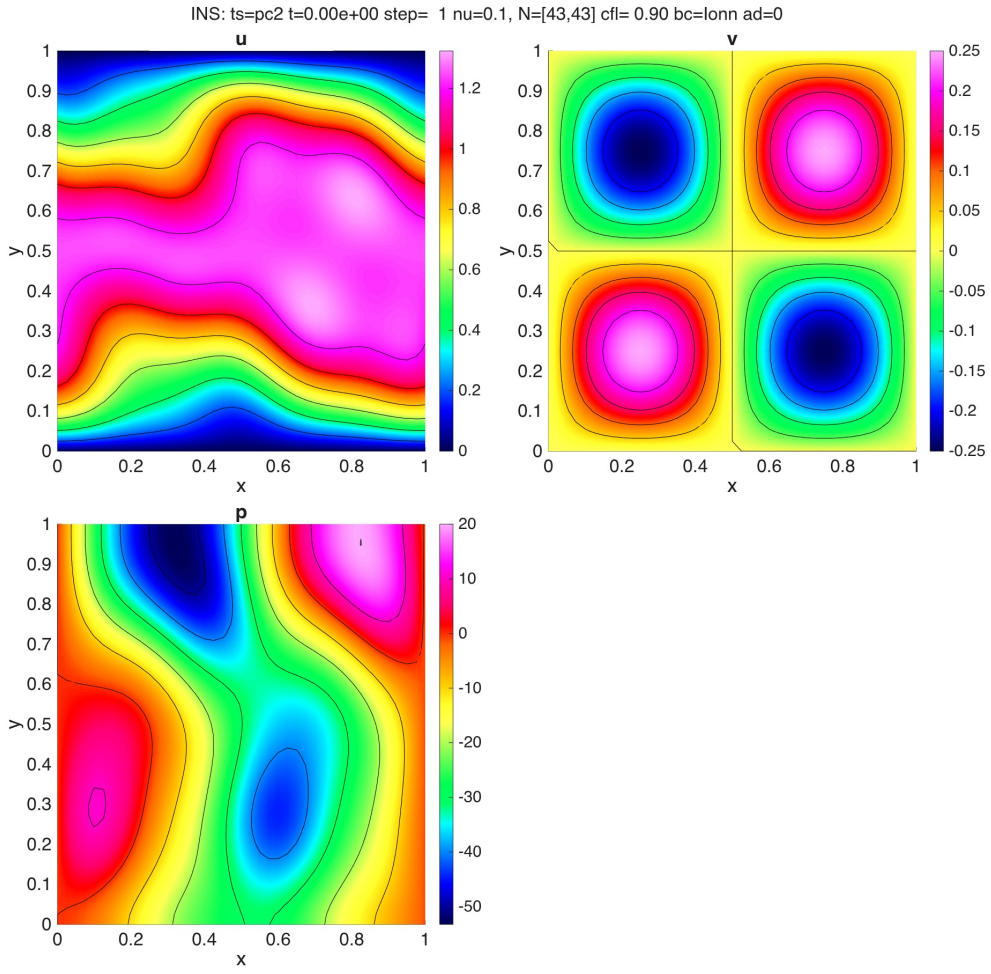
$$u(x, y, 0) = U(y) + \varepsilon u_p(x, y) \tag{25a}$$
$$v(x, y, 0) = \varepsilon v_p(x, y) \tag{25b}$$

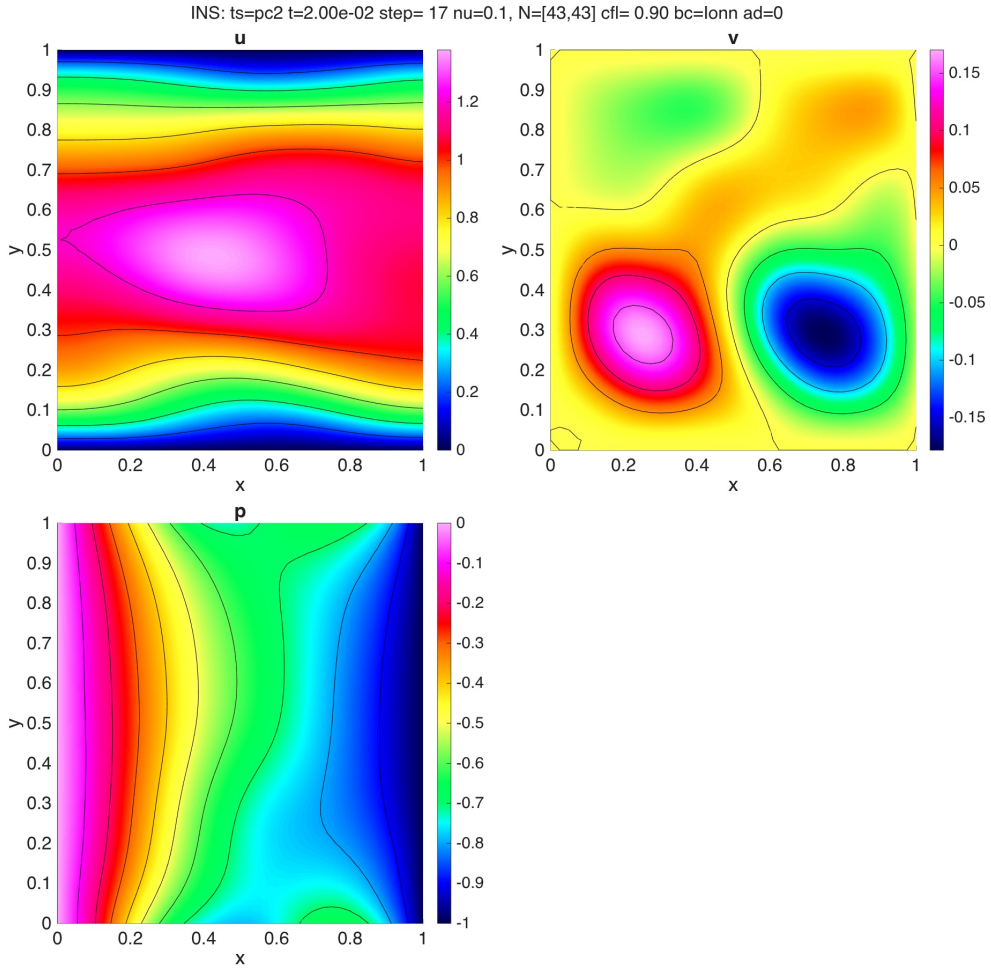
As our flow evolves, we should expect it to retain its form from (24) as time evolves. In the code this is tested with the following perturbation

$$u_p = (\cos^3(2\pi x) + \cos(2\pi(x + y))) \sin(2\pi y) \tag{26}$$
$$v_p = \sin(2\pi x) \sin(2\pi y) \tag{27}$$

Looking at the fluid evolve over a short time scale, we can see that the fluid quickly returns to the parabolic profile as desired:

```
ins -ts=pc2 -tf=0.02 -N0=40 -nu=0.1
  -ic=perturbedPoiseuille -perturbation=2.5e-1 -bcs=Ionn
  -pressureInflowValue=0 -uInflow=0 -outflowPressureCoeffp=1 -outflowPressureCoeffpn=0 -pOutflow=-1;
```




INS: ts=pc2 t=2.00e-02 step= 17 nu=0.1, N=[43,43] cfl= 0.90 bc=lonn ad=0
u
v
p
x
y

4.3. Travelling Wave of a Linearized Surface

Consider an irrotational fluid flow such that there exists a scalar function φ where

$$\mathbf{u} = \nabla\varphi \tag{28}$$

Then assuming that for a given wave number $k = 2\pi n/L$ that the free surface is a travelling wave of amplitude $0 < \eta_0 \ll 1$

$$\eta(x,t) = \eta_0 \cos(kx - \omega t) \tag{29}$$

We have that our equation for the velocity potential and dispersion relation is given by

$$\varphi(\mathbf{x},t) = \frac{g}{\omega}\left(1 + \frac{k^2L^2}{\mathrm{Bo}}\right)\frac{\cosh(ky+kH)}{\cosh(kH)} \cdot -\eta_0 \sin(kx - \omega t) \tag{30a}$$

$$\omega^2 = kg\left(1 + \frac{k^2L^2}{\mathrm{Bo}}\right)\tanh(kH) \tag{30b}$$

And the pressure of the fluid given by

$$p = p_0 - \rho(\varphi_t + gy) \tag{31}$$

```
        runGridConvergence -ts=pc2 -tf=1 -ms=none -knownSolution=GravityCapillaryWave -bcs=dddd -numResolutions=3 -grav
Running [ins -ts=pc2 -tzScale=1 -tf=1 -ms=none -knownSolution=GravityCapillaryWave -idebug=0 -nu=0.1 -bcs=dddd -N0=40 -
 ires=1: Nx=40 maxErr=[ 2.34e-10, 9.54e-11, 6.11e-11, 1.57e-09]
Running [ins -ts=pc2 -tzScale=1 -tf=1 -ms=none -knownSolution=GravityCapillaryWave -idebug=0 -nu=0.1 -bcs=dddd -N0=80 -
 ires=2: Nx=80 maxErr=[ 7.31e-11, 2.66e-11, 1.85e-11, 3.88e-10]
Running [ins -ts=pc2 -tzScale=1 -tf=1 -ms=none -knownSolution=GravityCapillaryWave -idebug=0 -nu=0.1 -bcs=dddd -N0=160
 ires=3: Nx=160 maxErr=[ 2.24e-11, 7.08e-12, 5.12e-12, 9.65e-11]
  N     dt      p-err  ratio   u-err  ratio   v-err  ratio    div   ratio    cpu(s)  ratio
 ---------------------------------------------------------------------------------------
  40  1.23e-03 2.34e-10       9.54e-11       6.11e-11       1.57e-09        1.96e+00
  80  3.08e-04 7.31e-11  3.2  2.66e-11  3.6  1.85e-11  3.3  3.88e-10  4.0   1.41e+01  7.2
 160  7.69e-05 2.24e-11  3.3  7.08e-12  3.8  5.12e-12  3.6  9.65e-11  4.0   2.32e+02 16.5
```

4.4. Taylor Green vortex

The Taylor Green vortex is an exact solution given by

$$u = +\alpha \sin(kx)\cos(ky)F(t) \tag{32a}$$

$$v = -\alpha \cos(kx)\sin(ky)F(t) \tag{32b}$$

$$p = \frac{\rho\alpha^2}{4}(\cos(2kx) + \cos(2ky))\, F^2(t) \tag{32c}$$

$$F(t) \stackrel{\text{def}}{=} \exp(-2\nu k^2 t) \tag{32d}$$

where α is a parameter defining the initial amplitude.

Figure 1 shows some results using the command

```
ins -ts=im2  -tf=.5 -ms=none -knownSolution=TaylorGreen -idebug=1 -nu=0.1 -bcs=nnnn -N0=100 -plotOption=1
```

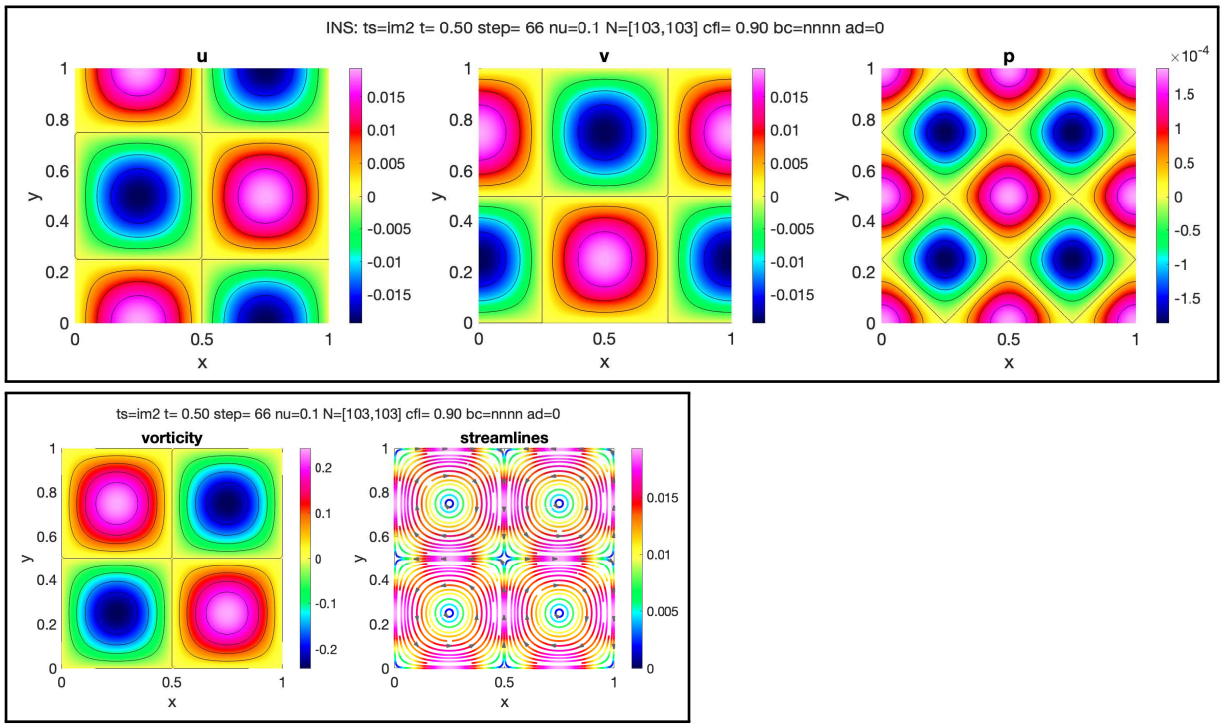


Figure 1: Taylor Green vortex

4.5. Shear flow in a periodic channel

Figure 2 shows the roll-up of a shear flow in periodic channel. The initial condition is perturbed shear flow

$$u(x, y, 0) = \tanh(\beta(y - y_m)), \tag{33}$$
$$v(x, y, 0) = \delta \cos(2\pi(y - y_m))) \sin(2\pi k_x x) \tag{34}$$

```
ins -ts=pc2 -tf=2.0 -tp=0.5 -ms=none -knownSolution=none -idebug=1 -nu=.0002 -ya=-.5 -yb=1.5 -xb=2
 -bcs=ppnn -N0=800 -ic=shear -movieMode=1 -plotOption=3 -plotErrors=0 -kx=1 -checkTimeStep=20
 -ad=1 -ad21=.1 -ad22=.1 -cfl=0.95
```

Artificial dissipation is added.

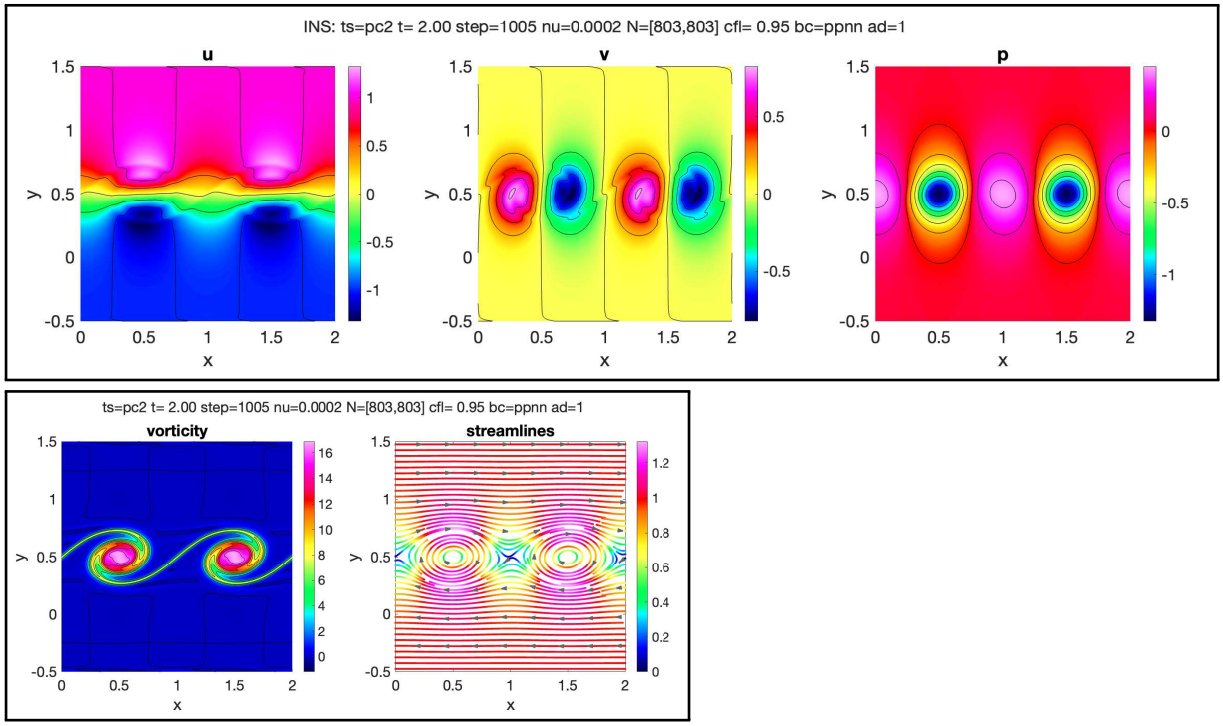


Figure 2: Rollup of a shear flow in a channel.

Here are the timings on a Mac-book pro showing that most of the time is spent in solving for the pressure.

```
  ------------- TIMINGS pc2 Nx=800 Ny=800 Nt= 1005 --------------------
                  cpu (s)     %
Total             5.89e+02  100.0
  setup           7.09e+01   12.0
  getUt           1.16e+02   19.7
  implicit solve  0.00e+00    0.0
  implicit factor 0.00e+00    0.0
  pressure solve  3.75e+02   63.7
  bc              3.50e-01    0.1
  plotting        2.03e+01    3.4
```

5. Viscous gravity-capillary travelling wave

The travelling wave solution of Section 4.3 is inviscid. Here we describe the viscous gravity-capillary travelling wave derived in Appendix C.2.1 of D. Serino's thesis (results in Section 4.5.1 of the thesis) and how the coefficients of the solution are computed in `src/getViscousCapillaryWaveCoeffs.m`.

5.1. Problem statement

Consider a fluid in the channel $\mathbf{x} \in [0, L] \times [-H, 0]$ with a no-slip wall at $y = -H$ and a free surface linearized about $y = 0$. Linearized travelling wave solutions are sought of the form

$$\mathbf{v}(x, y, t) = \hat{\mathbf{v}}(y) e^{i(kx-\omega t)}, \qquad p(x, y, t) = -\rho g y + \hat{p}(y) e^{i(kx-\omega t)}, \tag{35}$$

where $k = 2\pi n/L$ for a nonzero integer n, ω is a complex frequency, $g > 0$ is the gravitational acceleration and $\hat{\mathbf{v}}(y) = [\hat{v}_1(y), \hat{v}_2(y)]^T$, $\hat{p}(y)$ are coefficient functions. The physical solution is the real part of (35).
Substituting (35) into the linear Stokes equations gives

$$ik\hat{v}_1 + \hat{v}_2' = 0, \tag{36a}$$

$$-i\omega\rho\hat{v}_1 + ik\hat{p} = \mu(-k^2\hat{v}_1 + \hat{v}_1''), \tag{36b}$$

$$-i\omega\rho\hat{v}_2 + \hat{p}' = \mu(-k^2\hat{v}_2 + \hat{v}_2''), \tag{36c}$$

and taking the divergence gives $\hat{p}'' - k^2\hat{p} = 0$. The boundary conditions are no-slip at the bottom,

$$\hat{v}_1(-H) = \hat{v}_2(-H) = 0, \tag{37}$$

and the linearized free surface conditions at $y = 0$ (zero shear stress, and a normal stress balanced by gravity and surface tension),

$$\hat{\tau}_{12}(0) = \mu\big(\hat{v}_1'(0) + ik\hat{v}_2(0)\big) = 0, \qquad -\hat{p}(0) + 2\mu\hat{v}_2'(0) = -(\rho g + \gamma k^2)\,\hat{u}_2(0), \tag{38}$$

where $\hat{u}_2 = \hat{v}_2/(-i\omega)$ is the coefficient of the surface displacement. The gravity term comes from evaluating the hydrostatic pressure $-\rho g y$ at the displaced surface, and the surface tension term from the linearized curvature $\kappa \approx \eta_{xx}$. These conditions are consistent with the traction boundary condition (13a) used by the code.

5.2. Hyperbolic basis shifted to the bottom wall

The thesis writes the coefficient functions with the exponentials $e^{\pm ky}$ and $e^{\pm\alpha y}$. At the bottom wall these become e^{kH} and $e^{\alpha H}$ which are large, so the coefficients of e^{-ky} and $e^{-\alpha y}$ must be correspondingly small (in Table 4.1 of the thesis, B and D are about 10^{-5} times A and C). The resulting 4×4 system is poorly scaled, and this gets worse as ν decreases.

Instead we use hyperbolic functions of the distance from the bottom wall, $s = y + H \in [0, H]$. Let

$$\alpha^2 = k^2 - \frac{i\omega}{\nu}, \qquad q = \frac{ik}{\rho\omega}, \qquad G = \rho g + \gamma k^2. \tag{39}$$

Note: the sign of α^2 follows (C.27) of the thesis, which is consistent with (36); equation (4.75) of the thesis has the sign reversed. The pressure is $\hat{p} = a\cosh(ks) + b\sinh(ks)$ and, since $k^2 - \alpha^2 = i\omega/\nu$, the particular solution for $\hat{v}_2$ is $-q\big(a\sinh(ks) + b\cosh(ks)\big)$. The general solution is then

$$\hat{p}(s) = a\cosh(ks) + b\sinh(ks), \tag{40a}$$

$$\hat{v}_2(s) = -q\big(a\sinh(ks) + b\cosh(ks)\big) + c_1\cosh(\alpha s) + c_2\sinh(\alpha s), \tag{40b}$$

$$\hat{v}_1(s) = \frac{i}{k}\,\hat{v}_2'(s). \tag{40c}$$

At $s = 0$ we have cosh = 1 and sinh = 0, so the no-slip conditions (37) give c_1 and c_2 in closed form,

$$c_1 = q\,b, \qquad c_2 = q\,\frac{k}{\alpha}\,a. \tag{41}$$

Substituting, the velocity and its derivatives become

$$\hat{v}_2(s) = q\Big[a\Big(\frac{k}{\alpha}\sinh(\alpha s) - \sinh(ks)\Big) + b\big(\cosh(\alpha s) - \cosh(ks)\big)\Big], \tag{42a}$$

$$\hat{v}_2'(s) = q\Big[a\,k\big(\cosh(\alpha s) - \cosh(ks)\big) + b\big(\alpha\sinh(\alpha s) - k\sinh(ks)\big)\Big], \tag{42b}$$

$$\hat{v}_2''(s) = q\Big[a\,k\big(\alpha\sinh(\alpha s) - k\sinh(ks)\big) + b\big(\alpha^2\cosh(\alpha s) - k^2\cosh(ks)\big)\Big]. \tag{42c}$$

Each term vanishes at the wall, and only the two constants (a, b) remain.

5.3. Dispersion relation

Using $\hat{v}_1 = (i/k)\hat{v}_2'$, the free surface conditions (38) at $s = H$ become

$$\hat{v}_2''(H) + k^2\hat{v}_2(H) = 0, \qquad -\hat{p}(H) + 2\mu\hat{v}_2'(H) + \frac{iG}{\omega}\hat{v}_2(H) = 0. \tag{43}$$

This is a homogeneous 2×2 system $M(\omega)[a, b]^T = 0$. To keep the entries $O(1)$ for small ν, both rows are divided by $\cosh(\alpha H)$ (and the first row also by q). With

$$T_\alpha = \tanh(\alpha H), \qquad c_k = \frac{\cosh(kH)}{\cosh(\alpha H)}, \qquad s_k = \frac{\sinh(kH)}{\cosh(\alpha H)}, \tag{44}$$

the entries are

$$M_{11} = k\Big[\Big(\alpha + \frac{k^2}{\alpha}\Big)T_\alpha - 2k\,s_k\Big], \qquad M_{12} = \alpha^2 + k^2 - 2k^2c_k, \tag{45a}$$

$$M_{21} = -c_k + q\Big[2\mu k(1 - c_k) + \frac{iG}{\omega}\Big(\frac{k}{\alpha}T_\alpha - s_k\Big)\Big], \qquad M_{22} = -s_k + q\Big[2\mu(\alpha T_\alpha - ks_k) + \frac{iG}{\omega}(1 - c_k)\Big]. \tag{45b}$$

The dispersion relation is $\det M(\omega) = 0$. Some remarks on evaluating $M(\omega)$:

- $\cosh(kH)$ and $\sinh(kH)$ do not depend on ω and are computed once.
- Each evaluation needs only one complex exponential, $e^{-\alpha H}$, from which $T_\alpha = (1 - e^{-2\alpha H})/(1 + e^{-2\alpha H})$ and $1/\cosh(\alpha H) = 2e^{-\alpha H}/(1 + e^{-2\alpha H})$. Taking the principal square root, $\mathrm{Re}(\alpha) \geqslant 0$, keeps $e^{-\alpha H}$ bounded.
- All entries are even functions of α, so the branch of the square root does not affect $\det M$.

5.4. Solving for ω and the coefficients

The complex root of $\det M(\omega) = 0$ is found with a secant iteration in the complex plane (the Matlab function `fzero` only handles real functions). The initial guess is

$$\omega_0 = \sqrt{\Big(gk + \frac{\gamma k^3}{\rho}\Big)\tanh(kH)} \;-\; 2i\nu k^2, \tag{46}$$

which combines the inviscid dispersion relation for gravity-capillary waves on a fluid of depth H (the same relation used in Section 4.3) with the classical weak-viscosity decay rate $e^{-2\nu k^2 t}$ of a free surface wave (Lamb, *Hydrodynamics*). This guess ignores the boundary layer at the bottom wall and is only accurate for small ν. For the thesis parameters ($\nu = 0.05$, $k = 2\pi$, $\gamma = 0.05$) it gives $\omega_0 \approx 4.32 - 3.95i$ while the root in Table 4.1 is $\omega = 3.045 - 1.773i$. A warning is printed when the converged root differs from the default guess by more than 25%, and the user can supply a guess with the command line option `-omega0` (no spaces in the value), e.g.

```
ins -knownSolution=ViscousGravityCapillaryWave -omega0=3.045-1.773i ...
```

Given ω, the null vector $[a, b]^T$ is taken from the row of M with the larger norm, $[a, b]^T = [-M_{12}, M_{11}]^T$ (or the equivalent from the second row). The constants are then normalized so that the surface displacement is real at $t = 0$,

$$\hat{\eta} = \hat{u}_2(H) = \frac{i}{\omega}\hat{v}_2(H) = \eta_0, \qquad \eta(x, t) = \mathrm{Re}\big(\eta_0 e^{i(kx - \omega t)}\big), \tag{47}$$

where η_0 is the amplitude given by `-ampfs`. At $t = 0$ the surface is $\eta_0 \cos(kx)$, which matches the existing free surface initial condition `-icfs=cos`.

5.5. Comparison with the thesis constants

The thesis writes $\hat{p} = Ae^{ky} + Be^{-ky}$ and the homogeneous part of $\hat{v}_2$ as $Ce^{\alpha y} + De^{-\alpha y}$. These are related to our constants by

$$A = \frac{1}{2}(a+b)e^{kH}, \qquad B = \frac{1}{2}(a-b)e^{-kH}, \qquad C = \frac{1}{2}(c_1+c_2)e^{\alpha H}, \qquad D = \frac{1}{2}(c_1-c_2)e^{-\alpha H}. \tag{48}$$

The thesis normalizes $|\hat{\mathbf{u}}(0)| = u_0$ (equation (C.36)) but does not state the phase, so only ratios such as B/A, C/A, D/A can be compared directly with Table 4.1. In addition C and D swap if $\alpha \to -\alpha$. These values are printed when `-idebug` is positive.

5.6. Matlab implementation

The function

```
[par, vcw] = getViscousCapillaryWaveCoeffs( par )
```

uses `par.kx` (already scaled by 2π), `par.xa, par.xb, par.ya, par.yb`, `par.nu`, `par.rho`, `par.gravity` (the code uses `gravity`< 0 so $g = -$`par.gravity`), `par.gamma`, `par.ampfs` and `par.omega0` (0 means use the default guess (46)). It returns

`par.DispersionRelation` the complex frequency ω, which is used by the free surface initial condition in `getGrid.m`.

`vcw` a structure with the fields `omega, alpha, k, H, q, a, b, c1, c2` needed by `defineKnownSolution.m` to evaluate the exact solution (42), and `ABCD`, the thesis constants scaled by u_0, for checking against Table 4.1.

The thesis results in Table 4.1 use $L = H = 1$, $\nu = 0.05$, $k = 2\pi$, $\rho = 1$, $g = 1$ and $\gamma = 0.05$, 0.1, 0.2.

References

[1] G. S. Chesshire, W. D. Henshaw, Composite overlapping meshes for the solution of partial differential equations, JCP 90 (1) (1990) 1–64, publications/cgns1990.pdf.

[2] W. D. Henshaw, N. A. Petersson, A split-step scheme for the incompressible Navier-Stokes equations, in: M. M. Hafez (Ed.), Numerical Simulation of Incompressible Flows, World Scientific, 2003, pp. 108–125, publications/henshawPetersson2001.pdf.

[3] F. Meng, J. W. Banks, W. D. Henshaw, D. W. Schwendeman, Fourth-order accurate fractional-step IMEX schemes for the incompressible Navier-Stokes equations on moving overlapping grids, Computer Methods in Applied Mechanics and Engineering 366 (2020) 113040, publications/FourthOrderAccurateFractionalStepIMEXSchemeForINSOnMovingGrids_MengBanksHenshawSchwendeman2020.pdf.